

Visualizing Formal Software System Specifications Written in TLA+

Authors Name/s per 1st Affiliation (*Author*)

line 1 (of *Affiliation*): dept. name of organization
line 2: name of organization, acronyms acceptable
line 3: City, Country
line 4: e-mail: name@xyz.com

Authors Name/s per 2nd Affiliation (*Author*)

line 1 (of *Affiliation*): dept. name of organization
line 2: name of organization, acronyms acceptable
line 3: City, Country
line 4: e-mail: name@xyz.com

Abstract— Mission-critical systems have to comply to various formal standards – e.g. DO-178C and ISO26262 - about their operation, usually heavily relying on mathematical formalisms. These present many challenges to developers in terms of how to write, read and communicate the target system’s formal specifications. In most cases, having the right formal methods experts to write specifications does not solve the problem as the wider development team needs to be able to deeply understand the formal specifications. In this paper, we introduce a visualization tool that we have built on top of the TLC model checker for specifications written in the TLA+ formal domain specific language. Using our tool, the formally specified system behavior is visualized as a state machine where each state represents the system status (values of system variables), and transitions represent events that triggers system status update. The tool uses the output of the TLC which enumerates all possible states, augments it with the appropriate transitions and labels and visually renders it as a state machine.

Keywords—component; formal methods; TLA+; Visualization; (key words)

I. INTRODUCTION

Formal system specification has many advantages, especially for mission critical systems engineering. The specification is concise, precise and is amendable to a range of model checking and theorem proving-based formal analysis. This allows engineers to locate errors in the specification far more precisely – sometimes very challenging – than the code of the target system is based upon, compared to conventional testing techniques.

Previous research been using the PlusCal/TLA+ (Temporal Logic of Actions) approach [1-3] as a formal language that provides far greater expressiveness and reasoning capability than conventional software design modelling tools. TLA+ is a domain specific language for expressing formal properties of a system that is based on temporal logic. These specifications can be reasoned with to find a wide variety of errors. Code developed based on the formal specification is thus far more reliable and errors in the complex specifications can be located far earlier and proactively than post-implementation based testing techniques.

However, these models require engineers to have deep expertise in using temporal logic. For many systems engineers, these specifications and models are very hard to read and review, limiting the usability of the methods. We want to aid these engineers by applying visualization and model transformation techniques to translate formal systems’

specifications developed in TLA+ into much more user-friendly diagrams reflecting system behavior using one or more of the existing state-of-the-art diagramming techniques, such as state machine diagrams. Such visualization would enable highlighting pre-conditions and post-conditions, transitions, and states specified in TLA+ at different levels of abstractions. The visualizations are animated to “step through” a specification to help the engineers understand the inter-relationships between constructs, make sure that the model performs as the system is supposed to be, and to identify where problems in the specifications may occur.

We have developed a prototype TLA+ visualization and animation tool that takes a TLA+ specification, generates a state chart with transitions, allows an engineer to animate the specification by following transitions through the state chart, and view and zoom in parts of the visualization.

This paper is structured as follows. Section II gives a brief motivating scenario showing an example specification in TLA+. Section III provide an example state machine produced by our tool and snapshot from our visual debugger, as well as details on the architecture and workflow of the tool, TLAVisualizer. Section IV presents an example of a system formal specification and the produced state machine. Section V discusses the limitation of the current version of the tool and possible future directions.

II. MOTIVATION

We use a motivating example to demonstrate the need for our tool from Specifying Systems by Leslie Lamport [4]. This example formally specifies the behavior of HourClock that increments the system state variable *hr* by 1 every iteration until it reaches 12, and then it should go back to 1. Figure 1 shows the TLA+ specification of the HourClock module. The system state in this case is the variable *hr* which could take values 1 to 12.

```
1 ----- MODULE HourClock -----
2 EXTENDS Naturals
3 VARIABLE hr
4 HCini == hr \in (1 .. 12)
5 HCnxt == hr' = IF hr # 12 THEN hr + 1 ELSE 1
6 HC == HCini /\ []HCnxt_hr
7 -----
8 THEOREM HC => []HCini
9 =====
```

Figure 1. Example of TLA+ specification

While HourClock is a very simple and basic example of TLA+ specifications, to an untrained eye, it would be hard to comprehend. With larger more complex system

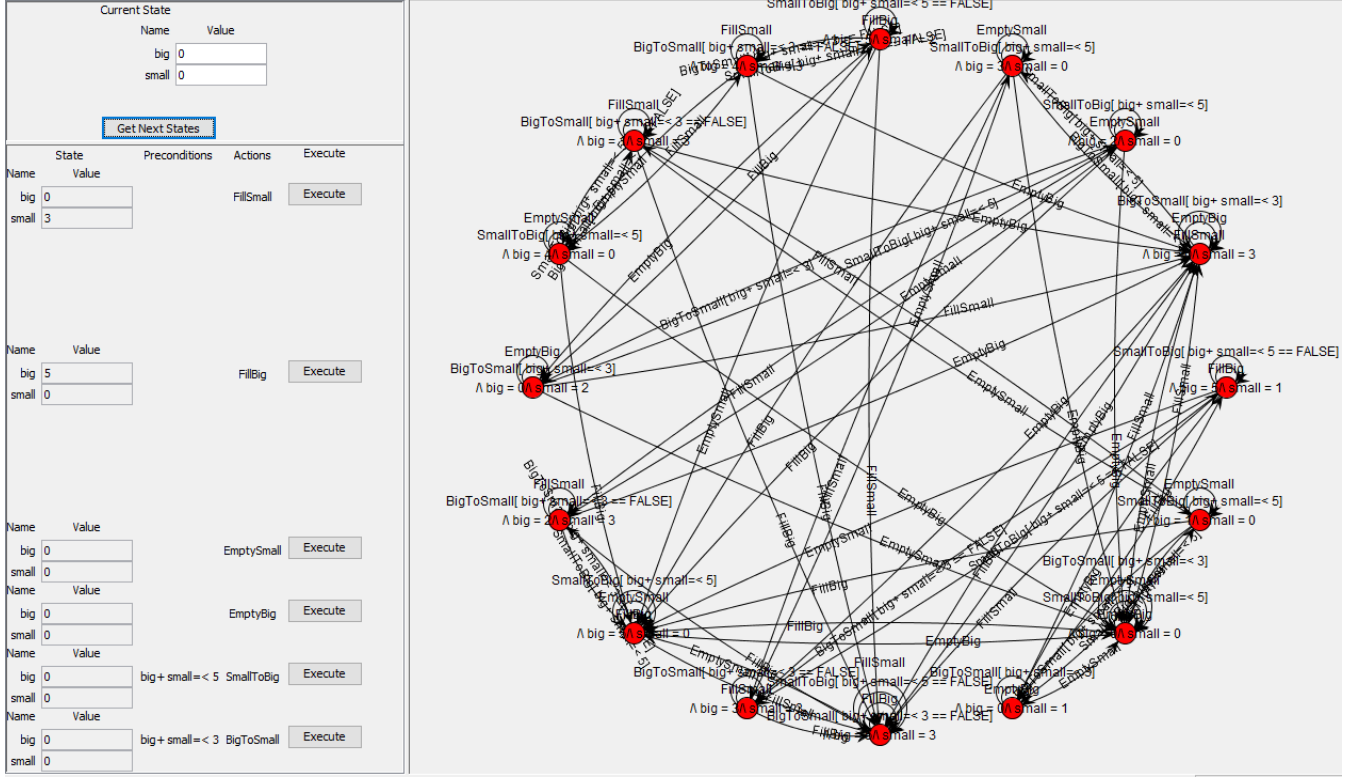


Figure 2. TLAVisualizer environment, demonstrating Die Hard problem¹.

specifications, it becomes very hard to understand and debug the TLA+ specifications [2, 5]. State machines on the other hand, have been used for decades and are well understood by practitioners and engineers alike [6, 7]. Consequently, we chose to use a state machine representation as the visual language of choice for visualizing and debugging TLA+ specifications. In the following section, we provide details of our tool, TLAVisualizer and briefly describe its architecture.

III. TLAVISUALIZER

Figure 2 shows our TLAVisualizer prototype tool in use and its environment, demonstrating Die Hard problem¹. The tool has four key components as shown in Figure 3:

TLA Parser: parses the model (.tla) file looking for predicates that together would make the state machine transitions (triggers) and conditions. At this point, we do not know the states of the model state machine.

Model Executor: This component is a wrapper on the existing TLA+ toolbox/model checker. It loads the model and its configuration file, and runs the TLA+ model checker component. It produces system states and transitions between these states which we extract for use in the visualizer.

Model Visualizer: This component produces a state chart-based graph of the model execution visualization model checker as a set of states and transitions using model execution results augmented with the parser identified predicates.

Model Debugger: This component allows system engineers to visually debug the specified system behavior. This includes:

- Stepping over states, showing what potential scenarios the modelled system could be in next given the current state,
- Visually picking a state to visit next and trigger the transition of the system to this new state,
- Visually setting up a system state and check what would the next steps be.

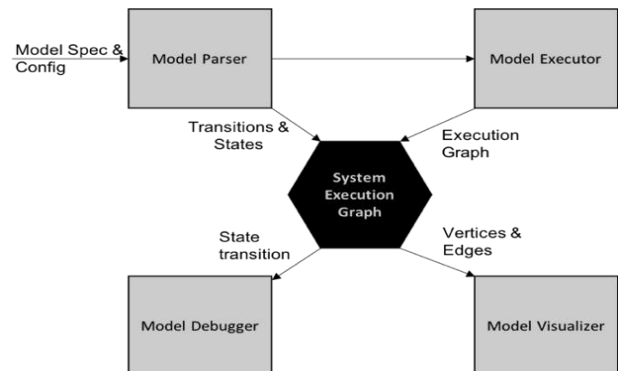


Figure 3. High level architecture and flow of the TLAVisualizer tool.

¹ <http://lampport.azurewebsites.net/video/videos.html>

IV. USAGE EXAMPLE

In this section we demonstrate TLAVisualizer's usage using two examples; the motivating example demonstrating the usage and debugging capabilities of the toolset, and an industry example.

A. HourClock System

Given the HourClock example of section II, Figure 4, demonstrates state machine representation of the same specification generated by our tool from the states produced by the TLC model checker. As shown in the figure, there are 12 states (values of the variable *hr*) and one transition event between states (*HCnxt*). It is also clear that there are no conditions necessary to transition from one state to the other, other than the event *HCnxt*.

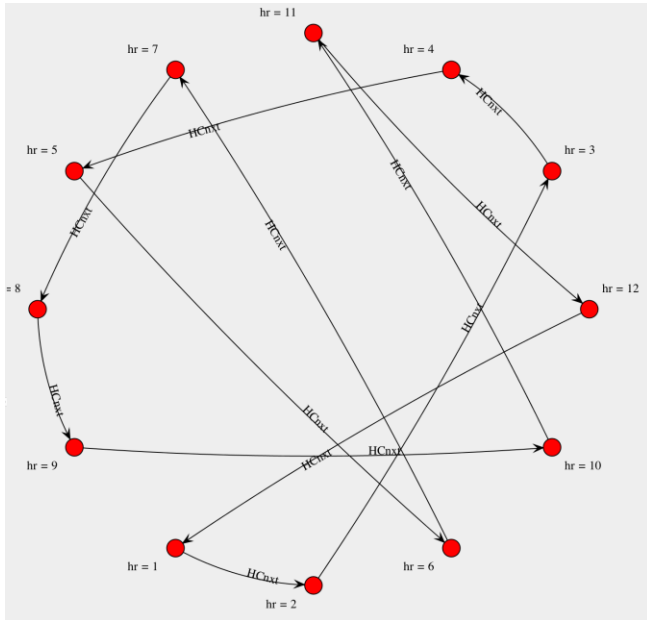


Figure 4. State Machine generated from the specification in Figure 1

TLAVisualizer provides facilities for users to debug and customize steps of a TLA+ specification. Figure 5 shows a snapshot from the tool's GUI where an end user can navigate between states and choose between different paths (visual debugger of the system behavior). The user will be presented with the current state of the system, and possible next states the system could be at (under different triggers and conditions). The user can choose one of the next transitions to execute, and the corresponding state machine visualization will be generated by TLAVisualizer. This would users to trace particular sections of the TLA+ specification visually, perhaps to examine for presence of bugs, or use for model checking.

As each state and transition is moved through, the visualization is animated by highlighting the appropriate state / transition link in the tool. Engineers can view the highlighted TLA+ specification elements linked to the highlighted state/transition to better understand which part of

the specification is impacted and drill down into potentially erroneous specification elements.

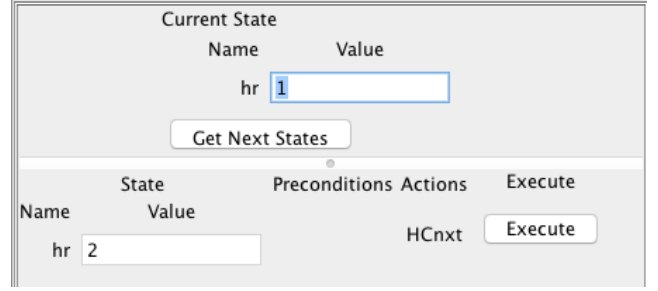


Figure 5. Stepping through specification with TLAVisualizer.

B. Surveillance System

As part of a large industry project, we wanted to model a surveillance system built up from a number of distributed sensors. This is a challenging system to both understand and debug due to the event-based and asynchronous nature of the code implementing the system. Here we describe a simplified and obfuscated version of this surveillance system and its visualization and debugging using TLAVisualizer.

At any given time, the system should be in one state: *Unknown*, *Running*, or *Failed*. When the surveillance system starts, it initially goes to the *Unknown* state. It then waits for messages from one of the sensors for 10 seconds. If a message is received during this time, it goes to *Running* state, otherwise if nothing happens it goes to *Failed* state.

The system can alternate between the *Running* and *Failed* states if no message is received in a ten-seconds window, it goes back to *Running* when a message received. Figure 6 shows a brief outline of the TLA+ model of the surveillance system.

```

2 EXTENDS Naturals
3 CONSTANTS UNKNOWN, OK, FAILED
4
5 VARIABLES state
6
7 report_received ==
8     ∧ state' = OK
9
10 timeout ==
11     ∧ state' = FAILED
12
13 Init ==
14     ∧ state = UNKNOWN
15
16 Next ==
17     ∨ report_received
18     ∨ timeout
19

```

Figure 6. Simplified TLA+ specifications of the Surveillance System

Figure 7, demonstrates the state machine visualization that was automatically generated by our TLAVisualizer tool. As can be seen from the figure, system states are captured by the state machines and their transitions are demonstrated. The engineer can use the TLAVisualizer debugger component to step through the specification via the state chart visualization

to understand how the specification handles incoming sensor events. They can also determine where the specification is erroneous in terms of handling e.g. out of sequence/unexpected/unknown events.

V. DISCUSSION

We have been using TLAVisualizer on our industry project that utilize TLA+-based event model specifications. TLAVisualizer has been very useful in visualizing behavior of the modelled system components and the visual debugging of system behavior. Engineers who struggle to understand the TLA+ DSL-based specification find the visualisation and animation greatly assists there understanding and use of the model. Additionally we have used our visual debugger to identify and correct several complex errors in the specifications.

At the same time, we have identified several limitations and possible extensions of the tool. The current version of the toolset does not support live edits, i.e. every time a user edits the configuration, or performs debugging by selecting specific states, the tool needs to execute the full specification first, and then generate the visualization. It also does not support concurrent edits by multiple engineers. For large projects this has proven to be time consuming.

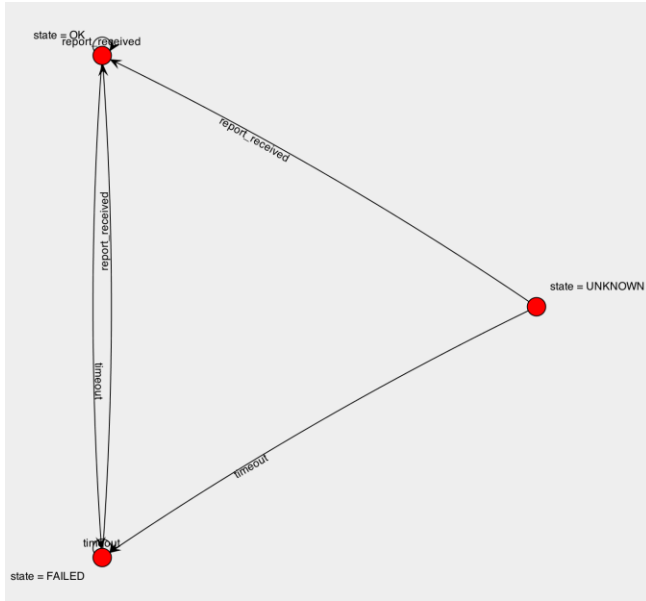


Figure 7. Surveillance state machine produced by TLAVisualizer

State explosion in model checking is well known problem for scalability. Such state explosion for complex specifications also has implications for our generated state machine visualizations. Such an example is demonstrated in Figure 8 for a very large TLA+ specification's state machine. While views and selective paths could address the problem, TLAVisualizer can benefit from clustering, better zoom-in/zoom-out and scaling of the generated state machine visualizations.

While web-based frameworks that are emerging to help with TLA+ debugging and usage examples are very useful,

they do not visualize the system states and transitions. Feedback from our industry engineers is that these still do not improve the understanding of complex modelled system behavior. In our tool we have made an assumption on how the model is written to extract transitions - states are just model variables, but we can in the future introduce an abstraction layer on top (group states that have the same meaning), which is currently supported in TLA+.

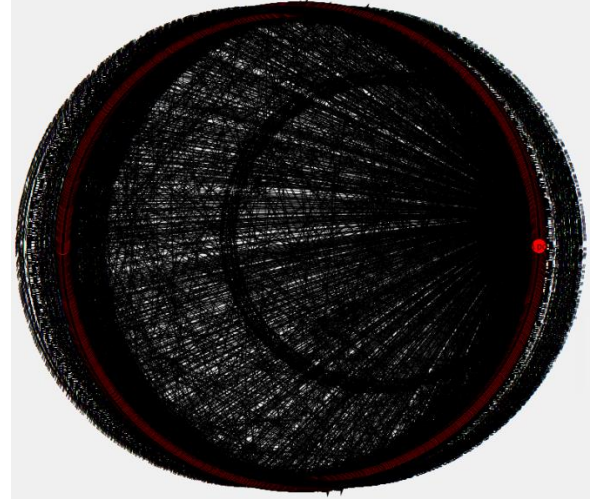


Figure 8. Example visualization of a very large system specification.

VI. CONCLUSION AND FUTURE WORK

Our experience deploying TLAVisualizer on industrial projects with systems engineers suggests that it provides a good initial step towards better user support for the use of TLA+ specifications. Currently, we are working on several improvements. For example, support for bi-directional Specification to and from visualizations, as currently visualizations are not interactive, i.e. users can only edit specifications. Another point of improvement is to provide more real-time visualization generation and collaborative editing of specification in the tool to allow multiple engineers to work on specification at the same time. Better support for viewing the visualizations to include zoom-in, zoom-out and brushing functionality for complex visualizations is also being worked on.

While the tool has been used within our group for many projects across broader IT domain, a proper usability study with candidate end users of the system needs to be performed.

REFERENCES

- [1] L. Lamport, "The PlusCal algorithm language." In International Colloquium on Theoretical Aspects of Computing, pp. 36-60. Springer, Berlin, Heidelberg, 2009.
- [2] Y. Yuan, M. Panagiotis, and L. Lamport. "Model Checking TLA+ Specifications." In Proceedings of the 10th IFIP WG 10.5 Advanced Research Working Conference on Correct Hardware Design and Verification Methods (CHARME '99), Springer-Verlag, London, UK, UK, 54-66, 1999.
- [3] S. Resch and M. Paulitsch, "Using TLA+ in the Development of a Safety-Critical Fault-Tolerant Middleware," IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Toulouse, pp. 146-152, IEEE, 2017.
- [4] L. Lamport, "Specifying Systems". Addison-Wesley, Boston, 2003.
- [5] Liu, Xuezheng, et al. "Using model checker and replay facility to debug complex distributed system." *Proceedings of the twentieth ACM symposium on Operating systems principles*. ACM, 2005.
- [6] Harel, David. "On visual formalisms." *Communications of the ACM* 31.5 (1988): 514-530.
- [7] Harel, David. "Statecharts: A visual formalism for complex systems." *Science of computer programming* 8.3 (1987): 231-274.